

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Title

Tesi di Laurea Magistrale in
Computer Science and Engineering - Ingegneria Informatica

Author: **Domenico Cacace**

Student ID: 000000
Advisor: Prof. Name Surname
Co-advisors: Name Surname, Name Surname
Academic Year: 2022-23

Abstract

Abstract EN

Keywords: here, the keywords, of your thesis

Introduzione

Abstract IT

Parole chiave: qui, vanno, le parole chiave, della tesi

Contents

Abstract **i**

Introduzione **iii**

Contents **v**

1 Introduction **1**

2 Fundamentals **3**

2.1 Quantum computing 3

2.1.1 Threats to classical cryptography 3

2.1.2 Post Quantum Cryptography 3

2.2 Code-based cryptography 3

2.2.1 General definitions 3

2.2.2 Difficult problems for cryptography 5

3 Side Channel Attacks **7**

4 Hamming Quasi-Cyclic **9**

5 Multiplication Algorithms **11**

5.1 Multiplication 101 11

5.1.1 Schoolbook multiplication 11

5.1.2 Integers and polynomials 11

5.1.3 Faster multiplication algorithms 12

5.2 FFT-Based multiplication 12

5.2.1 Convolutions and Fourier Transforms 13

5.2.2 Fast Fourier Transform 13

6 Empirical Evaluations **17**

7 Conclusions **19**

Bibliography **21**

A Appendix A **23**

List of Figures **25**

List of Tables **27**

Acknowledgements **29**

1 | Introduction

2 | Fundamentals

2.1. Quantum computing

Quantum computing is a *new* computing paradigm based on the theory of quantum mechanics.

2.1.1. Threats to classical cryptography

2.1.2. Post Quantum Cryptography

2.2. Code-based cryptography

In this section we give the basic concepts and definitions of coding theory ([MS77]), required to formally define the problems on which HQC and similar cryptosystems are based.

2.2.1. General definitions

We denote with $\mathbb{F}_2$ the binary finite field and with $\mathbb{Z}$ the integer ring.
We define $\mathcal{V} = \mathbb{F}_2^n$ the vector space of dimension n over $\mathbb{F}_2$ and $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$ the ring of binary polynomials of degree less than n for some $n \in \mathbb{N}$.
Additionally, we denote by $\mathtt{wt}(\cdot)$ the Hamming weight of a vector *i.e.* the number of its non-zero coordinates.

Definition 2.2.1 (Circulant matrix)**.** Let $\mathbf{v} = (v_0, \cdots, v_{n-1}) \in \mathcal{V}$. The circulant matrix induced by $\mathbf{v}$ is defined as a matrix whose i-th row is the vector $\mathbf{v}$ shifted by i positions to the right:

$$\mathbf{circ}(\mathbf{v}) = \begin{pmatrix} v_0 & v_1 & \cdots & v_{n-1} \\ v_{n-1} & v_0 & \cdots & v_{n-2} \\ \vdots & \vdots & \ddots & \vdots \\ v_1 & v_2 & \cdots & v_0 \end{pmatrix} \in \mathbb{F}_2^{n \times n}$$

The algebra of $n \times n$ circulant matrices over $\mathbb{F}_2$ is isomorphic to the algebra of polynomials in the ring $\mathcal{R}$:

$$\mathbf{circ}(\mathbf{v}) \simeq v_0 + v_1 x + v_2 x^2 + \cdots + x_{n-1} x^{n-1}$$

Definition 2.2.2 (Linear code)**.** A linear code $\mathcal{C}$ of lenght n and dimension k (denoted as $[n, k]$) is a subspace of $\mathcal{V}$ of dimension k; the elements of $\mathcal{C}$ are called *codewords*.

Definition 2.2.3 (Generator Matrix)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The matrix $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ is called the generator matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{mG}, \text{ for } \mathbf{m} \in \mathbb{F}_2^k\}$$

Definition 2.2.4 (Parity-Check Matrix)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ is called the parity-check matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{v} \in \mathbb{F}_2^n \text{ such that } \mathbf{H}\mathbf{v}^\top = 0\}$$

In other words, $\mathbf{H}$ is the generator matrix of the dual code $\mathcal{C}^\perp$.

Definition 2.2.5 (Syndrome)**.** Let $\mathcal{C} = [n, k]$ be a code, $\mathbf{H}$ its parity matrix and $\mathbf{v} \in \mathbb{F}_2^n$ a word. The syndrome of $\mathbf{v}$ is defined as

$$\sigma(\mathbf{v}) = \mathbf{H}\mathbf{v}^\top$$

From this definition, we can say that a vector $\mathbf{v}$ is a codeword if and only if its syndrome is the zero vector.

Definition 2.2.6 (Minimum distance)**.** Let $\mathcal{C} = [n, k]$ be a code and ω a norm on $\mathcal{V}$. The minimum distance of $\mathcal{C}$ is defined as

$$d = \min_{\mathbf{v}, \mathbf{w} \in \mathcal{C}, \mathbf{v} \neq \mathbf{w}} \omega(\mathbf{v} - \mathbf{w})$$

A code with minimum distance d ($\mathcal{C} = [n, k, d]$) can decode arbitrary codewords with at most $\delta = \lfloor \frac{d-1}{2} \rfloor$ errors.

Definition 2.2.7 (Quasi-cyclic Codes)**.** Let $\mathbf{c} = (\mathbf{c}_0, \cdots, \mathbf{c}_{s-1}) \in \mathbb{F}_2^{sn}$ be a vector, where we consider each $\mathbf{c}_i$ as a vector in $\mathbb{F}_2^n$. A linear code $\mathcal{C} = [sn, k, d]$ is Quasi-Cyclic of index s if, for any $\mathbf{c} \in \mathcal{C}$, the vector obtained by applying a circular shift to every block $\mathbf{c}_i$ simultaneously is also a codeword.

2.2.2. Difficult problems for cryptography

The problems we are interested in are all variants of the *decoding problem*, which requires to find the closest codeword to a given word with respect to a certain metric (the Hamming weight in our case).
In the case of linear codes we have shown that the decoding problem stays the same whether one is given the syndrome of the received vector or the vector itself; for this reason, we will only consider the *Syndrome Decoding* (SD) problem.

Definition 2.2.8 (SD Distribution)**.** Given $n, k, w \in \mathbb{N}$, the distribution $\mathsf{SD}(n, k, w)$ randomly samples a parity-check matrix $\mathbf{H} \xleftarrow{\$} \mathbb{F}_2^{(n-k)\times n}$, a vector $\mathbf{x} \xleftarrow{\$} \mathbb{F}_2^n$ such that $\mathtt{wt}(\mathbf{x}) = w$ and returns $(\mathbf{H}, \sigma(\mathbf{x}))$.

Definition 2.2.9 (SD Problem)**.** Given $(\mathbf{H}, \mathbf{y}^\top) \in \mathbb{F}_2^{(n-k)\times n} \times mathdsF_2^{(n-k)}$ from the $\mathsf{SD}(n, k, w)$ distribution, the SD problem requires to find a vector $\mathbf{x} \in \mathbb{F}_2^n$ such that $\sigma(\mathbf{x}) = \mathbf{y}^\top$ and $\mathtt{wt}(\mathbf{x}) = w$.

This problem has been proven to be NP-complete in [BMVT78].

Definition 2.2.10 (Decisional SD Problem)**.** Given $(\mathbf{H}, \mathbf{y}^\top) \in \mathbb{F}_2^{(n-k)\times n} \times mathdsF_2^{(n-k)}$, the DSD problem requires to determine whether $(\mathbf{H}, \mathbf{y}^\top)$ comes from the SD distribution or from the random distribution over $\mathbb{F}_2^{(n-k)\times n} \times mathdsF_2^{(n-k)}$.

3 | Side Channel Attacks

4 | Hamming Quasi-Cyclic

5 | Multiplication Algorithms

In this chapter etc etc

5.1. Multiplication 101

5.1.1. Schoolbook multiplication

5.1.2. Integers and polynomials

Multiply by value

If we think of the n digits of each input as coefficients of a polynomial, we can calculate the product by following these steps:

1. Sample the polynomials at $2n$ points
2. Perform the pointwise multiplication of the evaluations
3. Interpolate the result to obtain the coefficients of the product

Let $a = 123$ and $b = 45$ be two integers. We can calculate their product $c = 123 \cdot 45$

$$a = 123 \longrightarrow a(x) = 1x^2 + 2x + 3$$
$$b = 45 \longrightarrow b(x) = 0x^2 + 4x + 5$$

We sample the polynomials at $2n = 6$ points:

$$\begin{aligned} a(0) &= 1 \cdot 0^2 + 2 \cdot 0 + 3 = 3 & b(0) &= 0 \cdot 0^2 + 4 \cdot 0 + 5 = 5 \\ a(1) &= 1 \cdot 1^2 + 2 \cdot 1 + 3 = 6 & b(1) &= 0 \cdot 1^2 + 4 \cdot 1 + 5 = 9 \\ a(2) &= 1 \cdot 2^2 + 2 \cdot 2 + 3 = 11 & b(2) &= 0 \cdot 2^2 + 4 \cdot 2 + 5 = 13 \\ a(3) &= 1 \cdot 3^2 + 2 \cdot 3 + 3 = 18 & b(3) &= 0 \cdot 3^2 + 4 \cdot 3 + 5 = 17 \\ a(4) &= 1 \cdot 4^2 + 2 \cdot 4 + 3 = 27 & b(4) &= 0 \cdot 4^2 + 4 \cdot 4 + 5 = 21 \\ a(5) &= 1 \cdot 5^2 + 2 \cdot 5 + 3 = 38 & b(5) &= 0 \cdot 5^2 + 4 \cdot 5 + 5 = 25 \end{aligned}$$

The pointwise multiplication of the evaluations is:

$$\begin{aligned} c(0) &= a(0) \cdot b(0) = 3 \cdot 5 = 15 \\ c(1) &= a(1) \cdot b(1) = 6 \cdot 9 = 54 \\ c(2) &= a(2) \cdot b(2) = 11 \cdot 13 = 143 \\ c(3) &= a(3) \cdot b(3) = 18 \cdot 17 = 306 \\ c(4) &= a(4) \cdot b(4) = 27 \cdot 21 = 567 \\ c(5) &= a(5) \cdot b(5) = 38 \cdot 25 = 950 \end{aligned}$$

Finally, we can interpolate the result to obtain the coefficients of the product:

$$c(x) = 4x^3 + 13x^2 + 22x + 15$$

and convert back to an integer by shifting and adding the coefficients, finding that $123 \cdot 45 = 5535$.

This approach performs the pointwise multiplication in $\mathcal{O}(n)$, but requires $\mathcal{O}(n^2)$ operations to perform the sampling step and $\mathcal{O}(n^3)$ operations to interpolate the result, making it worse than the classical method.

5.1.3. Faster multiplication algorithms

5.2. FFT-Based multiplication

5.2.1. Convolutions and Fourier Transforms

We are however free to choose the points at which we sample the polynomials, which can drastically reduce the amount of operations needed. Instead of sampling at $2n$ random points, we can choose the points to be the N-th roots of unity, *i.e.* we calculate the Discrete Fourier Transform of the input polynomials:

$$\mathcal{F}(x) = \sum_{n=0}^{N} x_n e^{-\frac{i2\pi}{N}kn} \tag{5.1}$$

At this point we can perform the pointwise product in the *transformed* domain and, since $\mathcal{F}$ is invertible, apply the inverse transform:

$$\mathcal{F}^{-1}(X) = \frac{1}{N}\sum_{n=0}^{N} X_n e^{i\frac{2\pi}{N}kn} \tag{5.2}$$

The algorithms that compute the discrete Fourier Transform of a sequence (or its inverse) are called fast Fourier transforms (FFT), and have an asymptotic complexity of $\mathcal{O}(n \log n)$.

5.2.2. Fast Fourier Transform

Cooley-Tukey FFT

The Cooley-Tukey FFT [CT65] is the most common FFT algorithm. It is a *divide et impera* algorithm, which recursively expresses a DFT of size $N = N_1 N_2$ into N_1 smaller DFTs of size N_2 each. The simplest form of the Cooley-Tukey algorithm is the radix-2 decimation-in-time FFT, which computes the DFTs of the even and odd indexed inputs separately and then combines the results. Assuming that N is a power of 2, we can rewrite the DFT as:

$$\begin{aligned}
X_k &= \sum_{n=0}^{N-1} x_n e^{-\frac{2\pi i}{N}nk} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N}(2m)k} + \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N}(2m+1)k} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2}mk} + e^{-\frac{2\pi i}{N}k} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2}mk} \\
&= E_k + e^{-\frac{2\pi i}{N}k} O_k \qquad \text{for } k = 0, \cdots, N/2-1
\end{aligned}$$

where we split the DFT into two smaller DFTs, on the even and odd indexes respectively. Because of the periodicity of the complex exponential, we can also write:

$$\begin{aligned}
X_{k+N/2} &= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} m(k+N/2)} + e^{-\frac{2\pi i}{N}(k+N/2)} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} m(k+N/2)} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} mk} e^{-2\pi mi} + e^{-\frac{2\pi i}{N} k} e^{-\pi i} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} mk} e^{-2\pi mi} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} mk} - e^{-\frac{2\pi i}{N} k} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} mk} \\
&= E_k - e^{-\frac{2\pi i}{N} k} O_k \qquad \text{for } k = 0, \cdots, N/2-1
\end{aligned}$$

This means that we can express a DFT of size N as two DFTs of size $N/2$, where we can re-use intermediate results to calculate multiple outputs. Since the base case of the recursion has complexity $\mathcal{O}(n)$ we get the recurrence relation $T(n) = 2T(n/2) + \mathcal{O}(n)$; following the Master theorem [BHS80], the computational complexity of the Cooley-Tukey FFT is $\mathcal{O}(n \log n)$.

Cantor basis

In [Can89] Cantor showed how to get a special basis in which it is possible to compute the FFT efficiently for any $m = p^n$, with p a prime number; later in [GM10] Gao and Mateer used a simpler construction for the special case $p = 2$, where the Cantor basis $\mathbf{v} = (v_0, \cdots, v_{m-1})$ follows the structure:

$$v_0 = 1, \quad v_i^2 + v_i = v_i - 1 \text{ for } i \geq 1 \tag{5.3}$$

With respect to the Cantor basis $\mathbf{v}$, we define its sequence of subspaces as:

$$W_0 := \{0\}, \quad W_i := span\{v_0, \cdots, v_{i-1}\} \text{ for } i \geq 1 \tag{5.4}$$

By construction, we have that $\mathrm{d}W_0 \subset W_1 \subset \cdots \subset W_{m-1} = \mathbb{F}_{2^m}$ and the basis for a smaller space is contained into the basis of a larger space: this means that we can arbitrarily treat elements of smaller fields as elements in the larger fields by padding zeros to the extra dimensions.

With respect to each element of the basis v_i we can define its subspace polynomial $s_i(x)$

as:

$$s_i(x) = \prod_{a \in W_i} (x - a) \tag{5.5}$$

Since $\dim(W_i) = i$ we have $\deg(s_i(x)) = |V_i| = 2^i$. Such polynomials, as shown in [Can89, GM10], satisfy the following useful properties:

- linearity: $s_i(x + y) = s_i(x) + s_i(y)$
- recursivity: $s_{i+j}(x) = s_i(s_j(x))$

Segmentation of large polynomials

Suppose we are multiplying two binary polynomials $f(x)$ and $g(x) \in \mathbb{F}_2[x]$, represented as sequences of d bits:

$$a(x) = a_0 + a_1 x + \cdots + a_{d-1} x^{d-1}$$

In some cases, especially with high degree polynomials, it can be useful to split the polynomials into smaller segments, which can be easier to handle. The standard *Kronecker segmentation* for multiplying binary polynomials is performed as follows

1. Split the polynomials into $n = \lceil d/w \rceil$ w-bit blocks:

$$\begin{aligned} a(x) &= a_0 + a_1 x + \cdots + a_{d-1} x^{d-1} \\ &= (a_0 + \cdots + a_{w-1} x^{w-1}) + \cdots + (a_{d-w} + \cdots + a_{d-1} x^{w-1}) x^{w(n-1)} \end{aligned}$$

2. Define the field $\mathbb{F}_{2^{2w}} := \mathbb{F}_2[z]/g(z)$, with $g(z)$ irreducible polynomial of degree $2w$.
3. Define the map $\psi : \mathbb{F}_2[x] \to \mathbb{F}_{2^{2w}[y]}$ such that:

$$a'(y) = \psi(a(x)) := a'_0 + a'_1 y + \cdots + a'_{n-1} y^{n-1}$$

   where $a'_i = (a_{iw} + a_{iw+1} z + \cdots + a_{iw+(w-1)} z^{w-1})$.
4. Calculate the product $c'(y) = a'(y) \cdot b'(y) = c'_0 + c'_1 y + \cdots \in \mathbb{F}_{2^{2w}}[y]$
5. Map $z \mapsto x$ and $y \mapsto x^w$, collect terms and coefficients and find the result.

At step (4) we employ the FFT-based method to perform the pointwise multiplication.

6 | Empirical Evaluations

7 | Conclusions

Bibliography

[BHS80] Jon Louis Bentley, Dorothea Haken, and James B Saxe. A general method for solving divide-and-conquer recurrences. *ACM SIGACT News*, 12(3):36–44, 1980.

[BMVT78] Elwyn Berlekamp, Robert McEliece, and Henk Van Tilborg. On the inherent intractability of certain coding problems (corresp.). *IEEE Transactions on Information Theory*, 24(3):384–386, 1978.

[Can89] David G Cantor. On arithmetical algorithms over finite fields. *Journal of Combinatorial Theory, Series A*, 50(2):285–300, 1989.

[CT65] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of computation*, 19(90):297–301, 1965.

[GM10] Shuhong Gao and Todd Mateer. Additive fast fourier transforms over finite fields. *IEEE Transactions on Information Theory*, 56(12):6265–6272, 2010.

[MS77] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error correcting codes*, volume 16. Elsevier, 1977.

A | Appendix A

Appendix

List of Figures

List of Tables

Acknowledgements

Here you might want to acknowledge someone.